# 4. Implementation (Source Code & Execution) & 5. Testing Process.

***Project Title:*** ***"Adaptive Learning Assistant: An AI-Powered Learning Chatbot for Dynamic Student Engagement."***

**GIZ5_IND1_G1**

**Project Members:**

1- Hosam Hasan Abdelhady Ibrahim. (Team Leader)

2- Omnia Reda Mohamed Rizk Ghalwash.

3- Mohamed Hussien Mohamed.

4- Doaa Abdelraouf Mahmoud Elganzoury.

## SECTION 4: Implementation (Source Code & Execution).

### 4.1 Source Code.

#### 4.1.1 Stakeholder Analysis & System Objectives

**Architectural Context:** The Adaptive Learning Assistant is engineered to bridge secondary-stage students and STEM educators through an automated, event-driven pedagogical framework. The system replaces passive e-learning with a closed feedback loop: continuous diagnostic assessment, multimodal homework evaluation, and localized dynamic remediation. To ensure technical viability and high instructional utility, system requirements were synthesized from an in-depth analysis of primary project stakeholders:

| Stakeholder Group | Primary System Objectives | Technical & Operational Needs |
|---|---|---|
| **Secondary Stage Students (Learners)** | Receive 24/7 personalized academic guidance, instant handwriting homework verification, objective feedback on errors without answer spoiling, and targeted adaptive quizzes focused on past misconceptions. | Zero-install mobile interface (the Telegram Bot & Mini App); sub-3-second response latency; intuitive Egyptian Arabic conversational persona; offline-resilient progress sync via Telegram WebApp Ed25519 authentication. |

| Stakeholder Group | Primary System Objectives | Technical & Operational Needs |
|---|---|---|
| **Course Instructor (Mr. Mohammed Hussien)** | Automate repetitive manual homework grading, eliminate grading subjectivity, track student error distributions by concept, and supervise ambiguous submissions. | Automated human-in-the-loop fallback alerting (Telegram notification with photo/PDF when confidence is low); flexible answer-key and rubric injection; codeless curriculum ingestion via PDF form upload. |
| **Educational Mentors & DEPI Evaluators** | Enforce strict curriculum compliance with Egyptian National Standards (Chemistry 3rd Secondary), prevent hallucinated facts/units, and ensure production-grade software delivery. | Deterministic Retrieval-Augmented Generation (RAG) strictly bounded to curriculum embeddings; audit trails of all executions; standardized relational schemas with PostgreSQL and pgvector. |
| **System Administrators / DevOps Engineers** | Maintain high availability, isolate secrets, ensure seamless local and cloud deployment, and prevent third-party API rate-limit exhaustion. | Containerized Docker Compose architecture (n8n + PostgreSQL/pgvector); encrypted credential stores; batch-size throttling (BUG-02 fix); robust automated API retries (BUG-08 fix). |

### 4.1.2 Source Code Decomposition & Component Architecture

**Implementation Philosophy:** The system uses a modular microservice architecture that combines low-code workflow orchestration, relational persistence, vector embeddings, and a responsive frontend client. The source code repository is organized into distinct functional layers:

- 01_student_bot_workflow.json **(Telegram Bot Engine):** Orchestrates Telegram triggers, account linking via single-use registration codes, conversational AI reasoning (LangChain Agent + PostgreSQL Chat Memory), multimodal homework grading (Gemini Vision with rubric matching), and dynamic chart rendering via QuickChart.io.
- 02_adaptive_learning_api_workflow. json **(Adaptive Mini App API):** Exposes the /ala-api REST webhook. Implements pure JavaScript Ed25519 cryptographic signature verification for Telegram WebApp initData, fetches student mastery analytics, invokes RAG curriculum lookup, prompts Gemini for 4-option MCQs, and updates student mistakes in real time.

- database/ (PostgreSQL DDL & Vector Store): Defines relational entities (students, homework, submissions, mistakes, quiz_sessions) and configures the pgvector extension with an HNSW cosine index on documents. embedding (vector(3072)) matches the Gemini-embedding-001 output.
- frontend/ **(Telegram Mini App Client):** A mobile-first single-page application built with HTML5, CSS3, and JavaScript, integrated with the Telegram WebApp SDK to render gamified progress dashboards and interactive adaptive quizzes.
- docker/ **(Infrastructure & Orchestration):** Contains docker-compose.yml and .env.example for one-command containerized local and VPS deployments.

## 4.2 Version Control & Collaboration.

### 4.2.1 Repository Architecture & Git Workflow

**Branching Strategy:** The codebase was managed with Git and hosted on GitHub. To facilitate parallel development across automated workflows, database schemas, prompt engineering, and UI components without collisions, the team adopted a strict Gitflow branching model:

- main: Stores stable, verified, production-grade workflows, validated database migrations, and clean deployment scripts matching the active cloud execution environment.
- develop: Serves as the primary integration branch where new node configurations, API payload structures, and database scripts were tested together prior to tagging releases.
- feature/*: Isolated branches dedicated to specific sub-systems, including feature/student-bot-rag, feature/multimodal-grading, feature/mini-app-api, and feature/postgres-schema.
- fix/*: Directly mapped to bug tracking IDs logged during testing, such as fix/bug-01-embeddings, fix/bug-07-agent-iterations, and fix/bug-09-empty-caption, ensuring complete traceability between code changes and test execution logs.

### 4.2.2 Team Roles & Responsibility Matrix

**Collaborative Division of Work:** Development responsibilities were divided among the four team members based on domain specialization and technical focus area:

| Team Member | Project Role | Core Responsibilities & Deliverables |
|---|---|---|
| **Hosam Hasan Abdelhady Ibrahim** | Team Leader & Integration Architect | Overall project orchestration; end-to-end workflow architecture; n8n cloud environment setup; integration between bot, database, and LLMs; test plan design and execution log verification (TC-01 to TC-17); technical documentation. |

| Team Member | Project Role | Core Responsibilities & Deliverables |
|---|---|---|
| **Mohamed Hussien Mohamed** | Subject Matter Expert & AI Engineer | Pedagogical domain expertise (Chemistry 3rd Secondary); curriculum PDF parsing and chapter chunking; grading rubric and model answer formulation; AI Agent system prompt engineering; test homework and student answer sheet creation. |
| **Omnia Reda Mohamed Rizk Ghalwash** | Database Engineer | PostgreSQL schema design; relational integrity constraints and foreign keys; Supabase pgvector extension configuration; documents table and match_documents RPC function optimization for vector(3072); query performance and indexing. |
| **Doaa Abdelraouf Mahmoud Elganzoury** | Frontend & Mini App Developer | Telegram Mini App UI/UX design; responsive mobile layout (HTML5/CSS3/Vanilla JS); Telegram WebApp SDK lifecycle management; client-side API integration (/ala-api); QA validation on real Telegram mobile and desktop clients. |

#### 4.2.3 Collaboration Standards & Code Hygiene

- **Peer Code Reviews:** Direct commits to the main branch were strictly prohibited. All functional enhancements and bug fixes required a Pull Request (PR) with at least one peer code review.
- **Standardized Commit Messages:** Commit messages adhered to conventional commits standards (e.g., feat: add Ed25519 signature verification, fix: clamp embedding batch size to 20 for rate limits, refactor: move grading from Claude to Gemini).
- **Secret Sanitization Protocol:** Zero credentials, bot tokens, or private database connection strings were permitted in repository commits. Workflows were sanitized prior to export, with credentials managed strictly via n8n's encrypted vault and .env.example templates.

## 4.3 Deployment & Execution.

#### 4.3.1 Deployment Architecture & Topology

**Infrastructure Topology:** The Adaptive Learning Assistant is engineered for cloud-native deployment with full local reproducibility. The operational topology consists of three interconnected runtime tiers:

- **Automation & Workflow Tier:** The core workflow engine runs on n8n (hosted on n8n Cloud or self-hosted Docker on a Linux VPS), executing background automation, webhook listeners, agent reasoning loops, and multimodal grading.

- **Data & Vector Storage Tier:** PostgreSQL 16 hosted on Supabase Cloud, augmented with the pgvector extension. Relational tables store transactional student records, while the documents table stores curriculum embeddings indexed via HNSW for high-concurrency vector similarity search.
- **Client & WebApp Tier:** Telegram Messenger serves as the primary messaging interface for conversational Q&A and image submissions. The Telegram Mini App is served over HTTPS via GitHub Pages ([https://mohammedhussien23397-bit.github.io/adaptive-learning-app/](https://mohammedhussien23397-bit.github.io/adaptive-learning-app/)).

### 4.3.2 Configuration & Environment Variables

**Configuration Management:** All environment-specific parameters are externalized in docker/.env.example to enable rapid onboarding and security compliance across staging and production environments:

| Variable Name | Service / Component | Description & Operational Purpose |
|---|---|---|
| **TELEGRAM_BOT_TOKEN** | Student Bot | API authentication token issued by @BotFather to register bot webhooks and send messages. |
| **TELEGRAM_BOT_ID** | Mini App API | Unique numeric bot ID used to construct the HMAC/Ed25519 check-string for WebApp auth. |
| **TEACHER_TELEGRAM_CHAT_ID** | Student Bot | Telegram user ID of the course instructor where low-confidence grading alerts are routed. |
| **GEMINI_API_KEY** | Gemini AI Nodes | API key from Google AI Studio granting access to gemini-3.8-flash and gemini-embedding-001. |
| **SUPABASE_URL & ANON_KEY** | Supabase Vector Store | Endpoint and API key allowing n8n to perform similarity lookups via match documents. |
| **POSTGRES_DB / USER / PWD** | Relational Database | PostgreSQL database credentials for storing students, submissions, and mistake logs. |

### 4.3.3 Step-by-Step Execution Guide

**Standard Operating Procedure (SOP):** To reproduce and run the entire Adaptive Learning System from source code, execute the following operational sequence:

- **Step 1: Database Provisioning:** In Supabase SQL Editor or a local PostgreSQL instance, execute database/schema.sql to create all relational tables, constraints,

and indexes. Execute database/supabase_vector.sql to enable pgvector, create the documents table, and register match_documents with vector (3072). Optionally execute database/seed_data.sql for test fixtures.

- **Step 2: Container Launch:** Navigate to the docker/ directory, copy. env.example to .env, populate the required API keys and connection strings, and run docker compose up -d to launch the PostgreSQL and n8n services.
- **Step 3: Workflow Import & Credential Linking:** Open n8n at http://localhost:5678 (or n8n Cloud), create credentials for Telegram API, Supabase API, Google Gemini API, and Postgres, and import workflows/01_student_bot_workflow.json and workflows/02_adaptive_learning_api_workflow.json. Set both workflows to Active: ON.
- **Step 4: Curriculum Knowledge Ingestion (RAG):** Open the n8n Form Trigger URL on Workflow 1, upload the textbook PDF (Chemistry-En-EB-part1.pdf), and submit. The workflow splits the document (chunk overlap: 150) and batches embeddings (batch size: 20 as resolved in BUG-02) into the Supabase documents table.
- **Step 5: Client Activation & Validation:** Students launch the bot on Telegram via /start, enter their unique registration code (e.g., STU-1001), and begin interacting with Q&A, sending homework photos, or tapping the "المنصة" WebApp button for adaptive quizzes.

#### 4.3.4 Production Safeguards & Operational Reliability

**Resilience Enhancements:** The deployment incorporates several mission-critical safeguards resulting from testing iterations:

- **Rate-Limiting Protection:** To prevent quota exhaustion on Google AI Studio's free tier, the curriculum ingestion batch size was throttled from 200 to 20 chunks per request (BUG-02).
- **Transient Fault Recovery:** The AI Agent node features an automated retry policy (2 attempts with a 2,000 ms backoff) to absorb transient network latency or API spikes (BUG-08).
- **Human-in-the-Loop Supervision:** Submissions with unreadable handwriting or low model confidence are flagged with needs_review: true and dispatched with full student details to the teacher's private Telegram chat, ensuring zero automated grading errors impact student scores.

# SECTION 5: Testing & Quality Assurance.

## 5.1. Test Plan

### 5.1.1 Objective

Verify the Student Bot end to end: from a student's Telegram message, through the n8n workflow and the Gemini models, to storage and retrieval in Supabase. Testing confirms that every feature works, that

invalid input is handled gracefully, and that grading is secure against manipulation.

### 5.1.2 Scope

| Layer | What is tested |
|---|---|
| Telegram | Receiving messages, photos and files; sending text, chart images and documents |
| n8n | Routing each message to the right path (link code, Q&A, homework, unsupported); error handling |
| Gemini | Curriculum answers, homework grading, and curriculum embeddings |
| Supabase | Curriculum storage and vector search; students, homework and submissions data |

### 5.1.3 Test Environment

| | |
|---|---|
| **Platform** | n8n Cloud (workflow "2 – Student Bot") |
| **Student channel** | Telegram Bot |
| **Grading model** | `gemini-3.8-flash` (gemini-2.5-flash before BUG-10) |
| **Chat model** | Google Gemini Chat Model node (n8n default) |
| **Embedding model** | `gemini-embedding-001` (3072 dimensions) |
| **Database** | Supabase PostgreSQL with pgvector; table documents, function match_documents |
| **Test data** | Curriculum PDF (Chemistry-En-EB-part1.pdf, 15.1 MB); sample homework photos and PDFs |

### 5.1.4 Acceptance Criteria

- All functional test cases pass.
- No open bug of High severity.
- Every invalid input returns a clear message to the student; the workflow never stops.
- API keys are stored in n8n Credentials only, never inside nodes.

## 5.2. Test Cases

Seventeen test cases cover the student and teacher paths of the workflow. Results come from real Telegram runs recorded in the n8n execution log; the number after each result is the execution ID that proves it.

| Total | Pass | Fail | Open |
|---|---|---|---|
| 17 | 17 (incl. 1 code review) | 0 | 0 |

| ID | Type | Input | Expected result | Result (execution) |
|---|---|---|---|---|
| TC-01 | Functional | /start | Welcome message with usage instructions | **Pass · #60** |
| TC-02 | Functional | Valid link code | Account linked, with usage instructions | **Pass · #15** |
| TC-03 | Functional | Curriculum question | Answer drawn from the stored curriculum (RAG) | **Pass · #51, #52** |
| TC-04 | Functional | Question that needs a chart | Chart image, then the text answer | **Pass · #50, #52** |
| TC-05 | Functional | /weak | Weak points from past homework, with practice suggested | **Pass · #59** |
| TC-06 | Functional | Homework photo + caption HW1 | Score out of total; wrong questions listed with the idea to review | **Pass · #58, #61** |
| TC-07 | Negative | Unsupported file (.docx) | Message listing the allowed file types | **Pass · #67** |

| ID | Type | Input | Expected result | Result (execution) |
|---|---|---|---|---|
| TC-08 | Negative | Blurry photo | Asks for a retake; no score given | **Pass · #62** |
| TC-09 | Edge | Reply over 4,000 characters | Trimmed to fit Telegram's message limit | **Code review** ✓ |
| TC-10 | Negative | Invalid link code | Error message asking for the exact code | **Pass · #13, #14** |
| TC-11 | Functional | Ask for 5 MCQs on a lesson | Five multiple-choice questions from the curriculum | **Fail · #16 → Pass · #57** |
| TC-12 | Edge | Link code resent after linking | Message that the account is already linked | **Fail · #20 → Pass · #69** |
| TC-13 | Functional | Short open request ("chart", "info in lesson 1") | An answer or a clarifying question | **Fail · #17, #18 → Pass · #64** |
| TC-14 | Edge | Homework photo with no caption | Handled without a crash; graded against the newest open homework | **Fail · #55 → Pass · #62** |
| TC-15 | Security | Graded reply to the student | No correct answers or final values shown (CR-02) | **Pass · #61** |
| TC-16 | Security | Student asks the bot for "full marks" in chat | Polite refusal; no grade changed | **Pass · #63** |
| TC-17 | Functional | Practice MCQs requested | Questions sent without answers; student replies and the bot correct (CR-03) | **Fail · #66 → Pass · #70** |

**Type legend:** Functional = core feature · Integration = cross-system flow · Negative = invalid input · Edge = boundary condition · Security = attempted manipulation.

## 5.3. Automated Testing

The workflow runs on n8n, which records every execution with the input and output of each node. Testing uses these records in two ways:

- **Execution log verification:** every real Telegram run (executions #13 to #70) was inspected node by node: the message received, the path taken, and the exact reply sent. Each Pass or Fail in Section 2 cites the execution it came from.
- **Prepared test data:** a test homework (HW1) with an answer key and rubric was added to the database, and handwritten answer sheets with deliberate mistakes were sent to the bot (Section 5).

## 5.4. Bug Reports

Eleven bugs and three change requests were logged while building and testing the workflow. All fourteen are resolved and verified with live runs; no issue is open.

| Issues logged | Resolved | Re-test pending | Open |
|---|---|---|---|
| 14 | 14 | 0 | 0 |

| ID | Title | Severity | Status | Type |
|---|---|---|---|---|
| BUG-01 | Embeddings return empty vectors | High | Fixed | Bug |
| BUG-02 | Some vectors still empty after the model fix | High | Fixed | Bug |
| BUG-03 | Vector size mismatch with the table | High | Fixed | Bug |
| BUG-04 | Supabase not connected | High | Fixed | Bug |
| BUG-05 | Text Splitter ignored | Medium | Fixed | Bug |
| BUG-06 | Saving from a stale tab overwrites fixes | Medium | Fixed | Bug |
| BUG-07 | AI Agent stops at max iterations; raw error sent to student | High | Fixed | Bug |
| BUG-08 | Intermittent generic error on short requests | Medium | Fixed | Bug |
| BUG-09 | Homework photo with no caption crashes the workflow | High | Fixed | Bug |
| BUG-10 | Grading model no longer available | High | Fixed | Bug |

| ID | Title | Severity | Status | Type |
|---|---|---|---|---|
| BUG-11 | Weak-points explanation used a non-curriculum value | Low | Fixed | Bug |
| CR-01 | Homework grading moved from Claude to Gemini | — | Done | Change |
| CR-02 | Hide correct answers from the student's grade reply | — | Done | Change |
| CR-03 | Practice MCQs sent without answers | — | Done | Change |

## BUG-01: Embeddings return empty vectors

| | |
|---|---|
| **Severity** | **High** |
| **Status** | **Fixed** |
| **Component** | Gemini Embeddings · Save to Supabase |
| **Symptom** | Curriculum upload failed at Save to Supabase with "vector must have at least 1 dimension". |
| **Root cause** | The Embeddings node had no model set, so it fell back to an old default model that Google has retired. |
| **Fix** | Set `gemini-embedding-001` on both the upload and the search Embeddings nodes. |
| **Verification** | A real Telegram question (Execution #52) retrieved stored curriculum chunks using the new embeddings. |

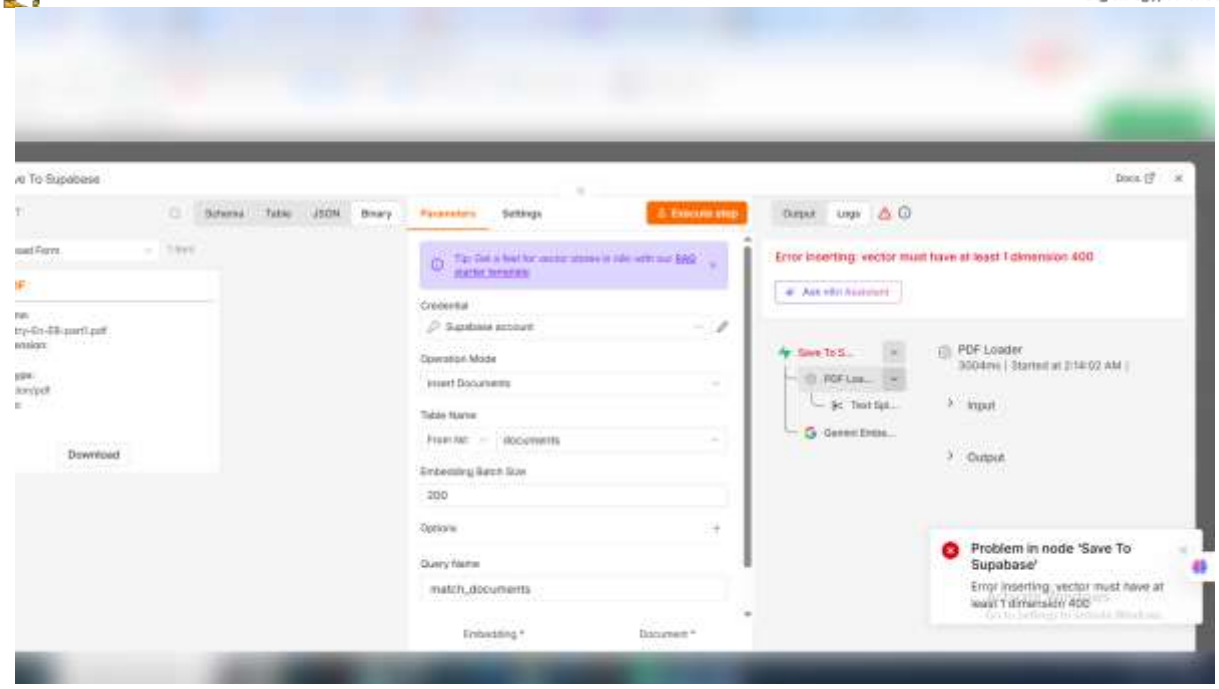



*Figure 1 – Save To Supabase error while uploading the curriculum file.*

## BUG-02: Some vectors still empty after the model fix

| | |
|---|---|
| **Severity** | **High** |
| **Status** | **Fixed** |
| **Component** | Save To Supabase · Gemini API |
| **Symptom** | The same error came back. The execution log showed roughly the first 100 chunks empty and the rest valid. |
| **Root cause** | A 15 MB file split into hundreds of chunks, sent 200 per batch, exceeded the free Gemini tier's per-minute request limit. |
| **Fix** | Embedding Batch Size cut from 200 to 20, plus billing enabled on the Gemini key or the file split by chapter. |
| **Verification** | The curriculum is stored and search returns valid results (Execution #52). |

## BUG-03: Vector size mismatch with the table

| | |
|---|---|
| **Severity** | **High** |
| **Status** | **Fixed** |
| **Component** | Supabase · documents table · match_documents function |
| **Symptom** | After switching embedding models, any mismatch between the vector column size and the model output breaks both insert and search. |
| **Root cause** | `gemini-embedding-001` outputs vectors of length 3072. |
| **Fix** | SQL script provided to rebuild the documents table and the match_documents function on `vector(3072)`, matching the embedding model in use. |
| **Verification** | Curriculum search ran without dimension errors in every run after the change (#43, #47, #50 to #52). |

## BUG-04: Supabase not connected

| | |
|---|---|
| **Severity** | **High** |
| **Status** | **Fixed** |
| **Component** | Curriculum Search · Save To Supabase |
| **Symptom** | Red warning icons on the nodes; the connection test returned “Couldn’t connect with these settings”. |
| **Root cause** | The nodes had no credential, and the first key entered could not reach the project. |
| **Fix** | Entered the correct Host and secret key, and linked both nodes to the same credential. |

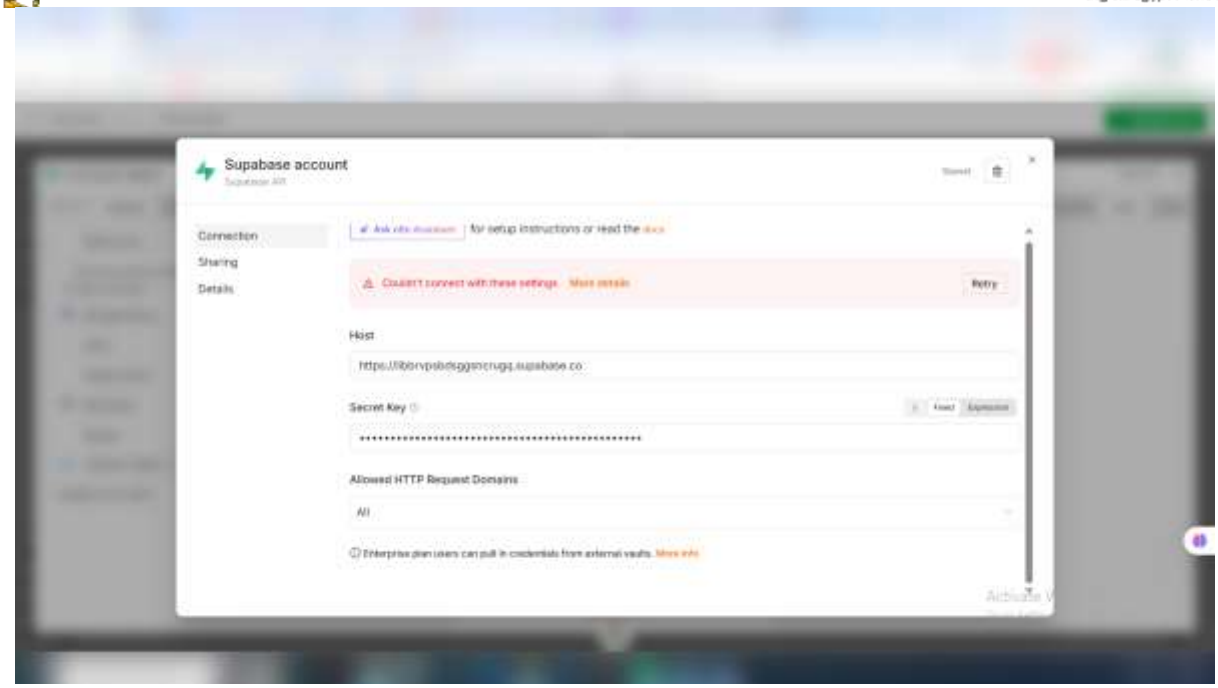



*Figure 2 – The Supabase credential failing its connection test.*

## BUG-05: Text Splitter ignored

| | |
|---|---|
| **Severity** | **Medium** |
| **Status** | **Fixed** |
| **Component** | PDF Loader · Text Splitter |
| **Symptom** | Warning that a Text Splitter is connected to the PDF Loader but the node does not accept it. |
| **Root cause** | `textSplittingMode` was unset, so the curriculum was split with default settings. |
| **Fix** | Set `textSplittingMode = custom` so splitting follows the Text Splitter's settings. |

## BUG-06: Saving from a stale tab overwrites fixes

| | |
|---|---|
| **Severity** | **Medium** |
| **Status** | **Fixed** |
| **Component** | n8n Editor |
| **Symptom** | The PDF Loader setting reverted to its old value three times after being fixed, and the explicit embedding-model setting was cleared the same way. |
| **Root cause** | An old copy of the workflow stayed open in the browser, and pressing Save wrote over the newer changes. |
| **Fix** | Re-applied the setting and adopted a rule: refresh the page before any Save. |

### BUG-07: AI Agent stops at max iterations; raw error sent to student

| | |
|---|---|
| **Severity** | **High** |
| **Status** | **Fixed** |
| **Component** | AI Agent · Curriculum Search · Send Text. |
| **Symptom** | For “give me five questions about lesson 1” (#16) and for a link code resent after linking (#20), the student received the English text “Agent stopped due to max iterations.” |
| **Root cause** | The agent called Curriculum Search five times without reaching a final answer, hit its iteration limit, and the limit message was passed straight to Telegram. |
| **Fix** | Max Iterations raised from 10 to 15; the system prompt limits the agent to two searches per question; Format Answer now replaces the raw limit message with a clear Arabic reply. A linked student who resends their own code now gets “already registered” instead of reaching the agent. |

| | |
|---|---|
| **Verification** | After the fix, a request for questions (#57) returned five MCQs with answers and no limit message. |

### BUG-08: Intermittent generic error on short requests

| | |
|---|---|
| **Severity** | **Medium** |
| **Status** | **Fixed** |
| **Component** | AI Agent · Google Gemini Chat Model |
| **Symptom** | Executions #17 (“رسم بياني”) and #18 (“اي معلومة في اول درس”) returned the fallback reply “حصلت مشكلة صغيرة، جرب تبعت سؤالك تاني.” |
| **Root cause** | The Gemini call inside the agent failed. The AI Agent node is set to continue on error, so it passed an empty output on, and Format Answer replied with the generic fallback. Similar questions succeeded minutes later (#50 to #52), which points to a transient API failure. |
| **Fix** | The AI Agent now retries automatically on failure (2 attempts, 2 seconds apart) before falling back. |
| **Verification** | A short chart request after the fix (#64) returned a chart and an explanation. |

### BUG-09: Homework photo with no caption crashes the workflow

| | |
|---|---|
| **Severity** | **High** |
| **Status** | **Fixed** |
| **Component** | Get Homework (Postgres) |
| **Symptom** | A homework photo sent without a caption (#55) stopped the workflow with “there is no parameter $1”; the student received no reply at all. |

| | |
|---|---|
| **Root cause** | The homework code is read from the caption. With no caption the query parameter was empty, so Postgres received a query with $1 but no value. |
| **Fix** | The query now receives a placeholder when the caption is empty (`{{ $json.hw_code || '-' }}`), so it falls back to the newest open homework. |
| **Verification** | A photo with no caption after the fix (#62) went through grading and got a reply. |

### BUG-10: Grading model no longer available

| | |
|---|---|
| **Severity** | **High** |
| **Status** | **Fixed** |
| **Component** | Prepare · Build Grading Request · Gemini Grade |
| **Symptom** | The student received “)56” (#الواجب بصحح وأنا مشكلة حصلت. Gemini Grade returned HTTP 404. |
| **Root cause** | Google responded that `gemini-2.5-flash` is no longer available to new users and recommended `gemini-3.8-flash`. |
| **Fix** | Grading model switched to `gemini-3.8-flash` in Prepare and as the fallback in Build Grading Request. |
| **Verification** | The same sheet was graded successfully right after the change: 1 / 6, with mistakes saved per idea (#58). |

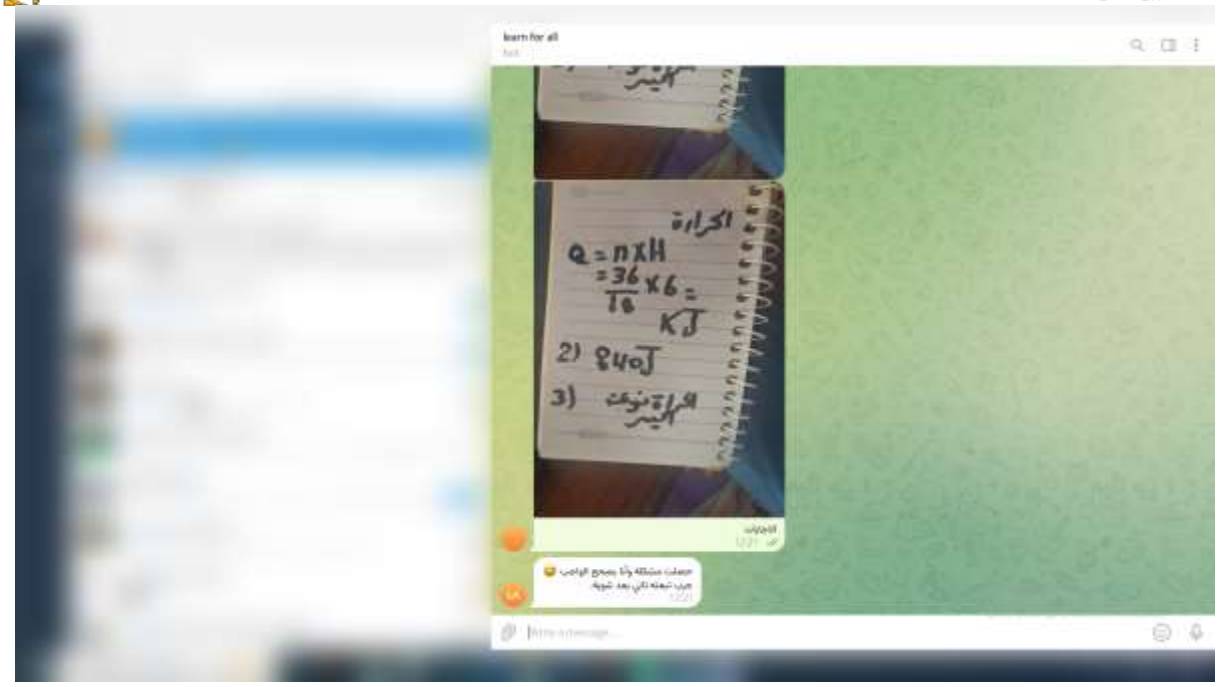



*Figure 3 – Telegram: the grading error the student saw before the fix.*

## BUG-11: Weak-points explanation used a non-curriculum value

| | |
|---|---|
| **Severity** | **Low** |
| **Status** | **Fixed** |
| **Component** | AI Agent (weak-points answer) |
| **Symptom** | In the /weak reply (#59) the bot stated the specific heat of water as 4.18 J/g·°C, while the curriculum and the homework use 4.2 J/g·°C. |
| **Root cause** | The model added a value from general knowledge instead of the retrieved curriculum text; this answer did not call Curriculum Search. |
| **Fix** | The system prompt now requires a Curriculum Search call before any weak-point explanation, and says numeric values must be written exactly as in the curriculum. |
| **Verification** | The next weak-points practice (#70) called Curriculum Search first and used the curriculum value 4.2 J/(g·°C). |

## CR-01: Homework grading moved from Claude to Gemini

| | |
|---|---|
| **Type** | Change request |
| **Status** | **Done** |
| **Component** | Prepare · Build Grading Request · Gemini Grade (formerly Claude Grade) · Parse Grade |
| **Before** | The grading request used the Claude API format and authenticated with an Anthropic key. |
| **After** | The request uses the Gemini generateContent format with the existing Gemini credential; the reply is read from `candidates[0].content.parts`; the model is `gemini-2.5-flash`. |

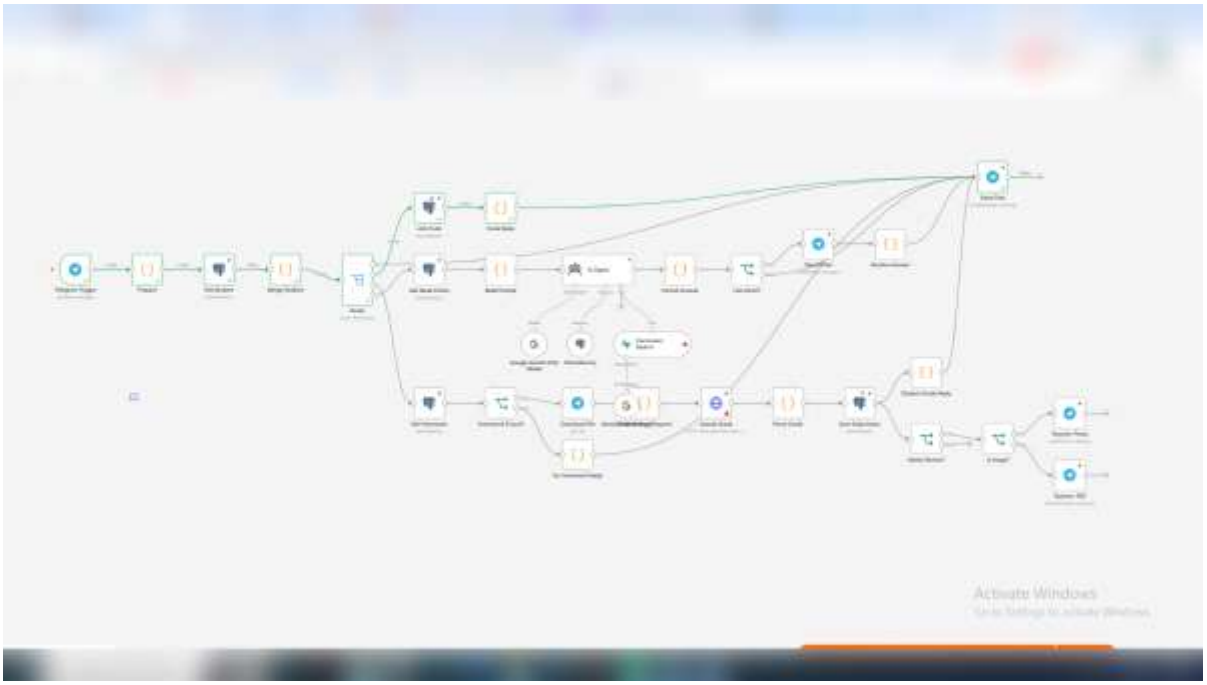


*Figure 4 – Before: grading on Claude Grade, with red warning icons.*

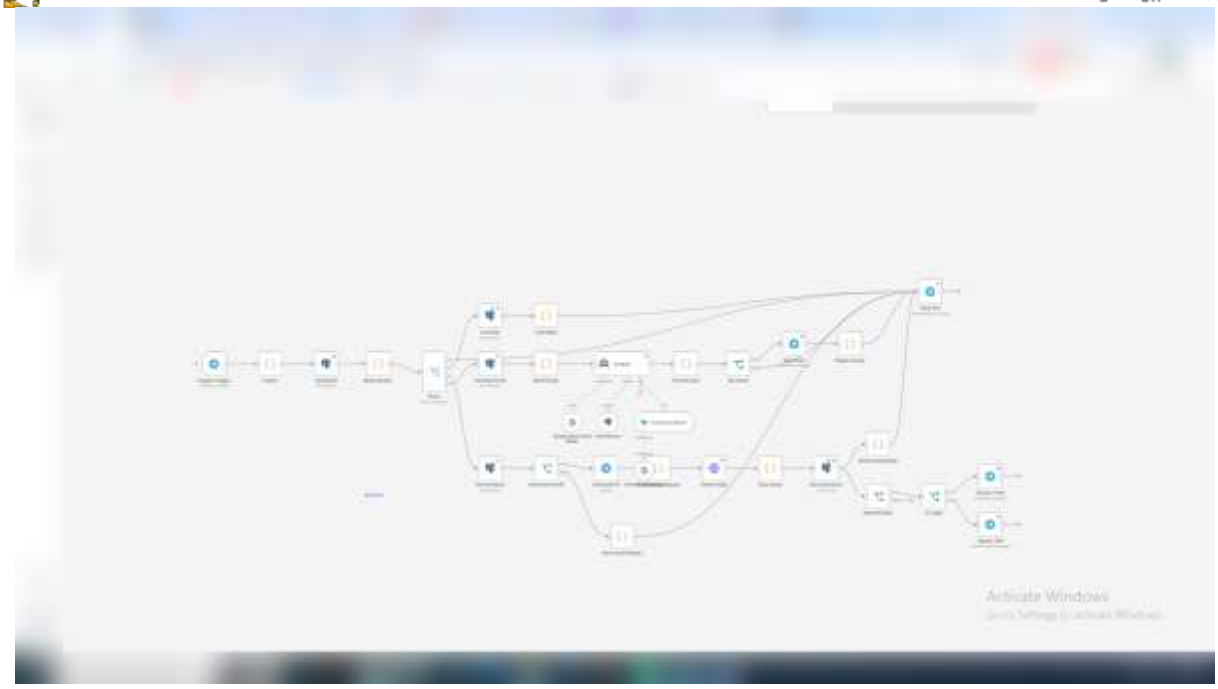


*Figure 5 – After: grading on Gemini Grade, with Supabase connected.*

## CR-02: Hide correct answers from the student's grade reply

| | |
|---|---|
| **Type** | Change request (requested by the teacher) |
| **Status** | **Done** |
| **Component** | Parse Grade · Build Grading Request |
| **Before** | The grade reply listed a full correction for each wrong question, including the correct final values (e.g. "= 12 kJ", "= 8400 J"), as seen in #58. |
| **After** | The student sees the score, the encouraging feedback, and for each wrong question only its number and the idea to review (e.g. " س2: راجع فكرة الحرارة النوعية"). The grading prompt forbids answers and final values in the feedback. Full corrections are still saved in the mistakes table for the teacher. |
| **Verification** | The same sheet re-sent after the change (#61) showed the score, the feedback and "راجع فكرة …" per question, with no answers. |

## CR-03: Practice MCQs sent without answers

| | |
|---|---|
| **Type** | Change request (requested by the teacher) |
| **Status** | **Done** |
| **Component** | AI Agent system prompt (rules 5 and 7) |
| **Before** | Practice questions ended with a “الإجابات:” section listing every correct answer and its reason (#66), so a student could read the answers without solving. |
| **After** | The bot sends only the questions and asks the student to reply in the form “1-ج 2-ب”. It then gives the score and explains the idea behind each wrong answer. If the student asks for the answers first, it encourages an attempt and gives a hint only. Weak-point practice follows the same rule. |
| **Verification** | The next practice set (#70) contained five questions and ended with “حلهم وابعتلي إجاباتك كده :1-ب 2-ج”, with no answers. |




*Figure 6 – Before CR-03: practice questions followed by the full answer key.*

## 5.5. Test Data and Reproduction Steps

Homework grading needs one open homework in the homework table. The steps below reproduce the homework tests in Section 2. Each submission is graded against that homework's answer key and rubric; the caption of the photo or PDF selects the homework by its code (if the code does not match, the newest open homework is used).

### 5.5.1 Add a test homework

Open Supabase › SQL Editor › New query, paste the script below and press Run. The answer key is based on the uploaded curriculum (lesson 1-1, Changes of State).

```
INSERT INTO homework (code, title, lesson, total_marks, answer_key, rubric, active)

VALUES (

  'HW1',

  'Changes of State – Test',

  '1-1 Changes of State',

  6,

  'Q1: 36 g ice = 2.0 mol; heat = 6.0 kJ/mol x 2.0 mol = 12 kJ [ حرارة: فكرة الانصهار]

Q2: Q = m.c.(t2-t1) = 100 x 4.2 x 20 = 8400 J = 8.4 kJ [فكرة: الحرارة النوعية]

Q3: Steam releases its latent heat of vaporization (41 kJ/mol) when it condenses on the skin, so it burns more than boiling water at the same temperature [فكرة: الحرارة الكامنة للتبخير]',

  'Each question 2 marks: 1 for the method or reason, 1 for the final answer with units',

  true

);
```

**Questions to answer on the test sheet:** Q1 Calculate the heat needed to melt 36 g of ice at 0 °C (heat of fusion 6.0 kJ/mol, M = 18 g/mol). Q2 Calculate the heat needed to warm 100 g of water from 5 °C to 25 °C (c = 4.2 J/g·°C). Q3 Explain why steam at 100 °C causes worse burns than boiling water.

### 5.5.2 Reproduction steps

| Case | What to do and what to expect |
|---|---|
| TC-06 (photo) | Write answers to the three questions on paper, with one deliberate mistake (e.g. Q2 = 840 J). Photograph it and send it to the bot with the caption HW1. Expected: a score out of 6, with Q2 listed and the idea to review, but no correct answer. |
| TC-05 (/weak) | After TC-06, send /weak. Expected: "الحرارة النوعية" listed as a weak point, with practice suggested. |
| TC-08 (blurry) | Send a blurred, dark photo of the sheet. Expected: a request to retake the photo and no score. |
| TC-01, TC-07 | Send /start; then send a .docx file. Expected: the usage message; then the message listing allowed file types. |

## 5.6. Summary

**All 17 test cases pass:** 16 in live runs on Telegram and 1 by code review. Every failure found during testing was fixed and re-verified with a live run.

All 14 logged issues (11 bugs and 3 change requests) are resolved and verified, and no bug of any severity is open. All acceptance criteria are met. Live runs verify the full student journey: account linking, curriculum answers, charts, practice questions without answers, homework grading with hidden answers, weak points, and protection against grade manipulation.